

Software & Setup Guide

Which software each person installs, how to install it, and how to use it day to day. **Everything here is free.** No licence, no credit card, no trial that expires mid-project.

0. Read This First

Three rules that will save you weeks.

- Everyone installs the shared toolchain (Section 1). Only your own person-specific tools after that.** Do not install another person's stack "just in case" — it fills your disk and creates version conflicts you will debug at 2 a.m.
- Match the versions in this document exactly.** "It works on my machine" is almost always a version mismatch. Pin versions on day one.
- If your laptop is weak (4 GB RAM, no dedicated GPU), use the cloud alternatives in Section 6.** They are free and often faster than running Docker locally.

Windows tip used throughout this guide: if a `winget install <ID>` command ever fails with "no package found", run `winget search <name>` to get the correct ID. Package IDs change occasionally.

1. Shared Toolchain — All Four People Install These

Do this on **Day 1, Week 1**, before any other work.

Tool	Version	What it is for	Install (PowerShell, as Administrator)
Git	2.45+	Version control. Every line of code goes through it.	<code>winget install Git.Git</code>
Node.js	20 LTS	Runs the backend and builds the frontend.	<code>winget install OpenJS.NodeJS.LTS</code>
pnpm	9+	Package manager. Faster than npm and handles our monorepo properly.	<code>npm install -g pnpm</code>
VS Code	latest	The editor everyone uses. Shared settings = shared formatting.	<code>winget install Microsoft.VisualStudioCode</code>
Docker Desktop	latest	Runs the database, cache and message queue without installing them natively.	<code>winget install Docker.DockerDesktop</code>
Postman	latest	Test APIs without writing a UI. P1 publishes the collection; everyone uses it.	<code>winget install Postman.Postman</code>
Windows Terminal	latest	A terminal that does not fight you.	<code>winget install Microsoft.WindowsTerminal</code>

Verify everything worked — run this and confirm you get version numbers, not errors:

```
git --version
node --version
pnpm --version
docker --version
```

VS Code extensions (everyone)

Install from the Extensions panel (`Ctrl+Shift+X`):

Extension	Why
ESLint + Prettier	Auto-format on save. Ends all formatting arguments permanently.
GitLens	See who wrote each line and why. Essential during code review.
Error Lens	Shows errors inline instead of hidden in a panel.

Extension	Why
Markdown Preview Mermaid	Renders the diagrams in this project's documentation.
Docker	Start and stop containers without leaving the editor.
Thunder Client	A lightweight Postman alternative inside VS Code.

Commit a shared `.vscode/settings.json` to the repository so all four editors behave identically:

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit" },
  "files.eol": "\n"
}
```

`"files.eol": "\n"` matters more than it looks. Without it, Windows line endings make every file appear 100% changed in every pull request, and code review becomes impossible.

2. P1 — Backend & Database Lead

You build: server services, the PostgreSQL database, all APIs, authentication, booking and dispatch.

Install

Tool	Version	Purpose	Install
PostgreSQL	16	The main database.	<code>winget install PostgreSQL.PostgreSQL.16</code>
PostGIS	3.4	Geospatial extension — "find mechanics within 5 km". Comes with the Stack Builder in the PostgreSQL installer.	via PostgreSQL Stack Builder
DBeaver Community	latest	Visual database client. Browse tables, run queries, view query plans.	<code>winget install DBeaver.DBeaver.Community</code>
NestJS CLI	11	Generates services, controllers and modules so you write structure, not boilerplate.	<code>npm i -g @nestjs/cli</code>
Redis + Kafka	—	Cache and event bus. Do not install natively — run them in Docker.	see below
dbdiagram.io	web	Draw the ER diagram for Review 1. Free, no install.	dbdiagram.io

The one command that starts your whole backend

Create `docker-compose.yml` in the project root and run `docker compose up -d`. This gives you Postgres with PostGIS, Redis and Kafka in about 60 seconds — no native installs, and identical on all four machines.

```
services:
  db:
    image: postgis/postgis:16-3.4
    environment:
      POSTGRES_PASSWORD: devpassword
      POSTGRES_DB: roadassist
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
  kafka:
    image: redpandadata/redpanda:latest
    command: redpanda start --overprovisioned --smp 1 --memory 1G
    ports: ["9092:9092"]
```

```
volumes:
  pgdata:
```

Your daily workflow

Step	Command
1. Start infrastructure	<code>docker compose up -d</code>
2. Apply database migrations	<code>pnpm db:migrate</code>
3. Load test data	<code>pnpm db:seed</code>
4. Run the backend	<code>pnpm dev</code>
5. Test an endpoint	Postman, or <code>curl http://localhost:3000/v1/health</code>
6. Run tests before pushing	<code>pnpm test</code>

How to actually use these for your reviews

- **Review 1 (ER diagram):** design in [dbdiagram.io](#), export the PNG for your slides, then write the real schema as migration files. Never hand-draw a schema you have not run.
- **Review 1 (proving indexes work):** in DBeaver, run `EXPLAIN ANALYZE` before and after adding an index. Screenshot both. That comparison *is* your demo.
- **Review 2 (140 APIs):** let NestJS generate the OpenAPI specification automatically — never write API documentation by hand, it always goes stale. Export the Postman collection and share it so P2 can build against it.
- **Review 3 (load testing):** P4 runs k6 against your endpoints. Use `pg_stat_statements` in DBeaver to find the slow queries it exposes.

3. P2 — Frontend & Mobile Lead

You build: the mobile app, the web portals, the design system, the offline client.

Install

Tool	Version	Purpose	Install
Android Studio	latest	Android SDK, emulator, and device debugging. Large — about 8 GB.	<code>winget install Google.AndroidStudio</code>
Java JDK	17	Required to build Android.	<code>winget install Microsoft.OpenJDK.17</code>
Expo CLI	latest	Run the app on your real phone instantly, no cable, no emulator.	<code>npm i -g expo</code>
Expo Go	app	Install on your Android phone from the Play Store. Scan a QR code and the app runs.	Play Store
Figma	web	Design screens, build the design system, share with the team. Free tier is enough.	figma.com
React DevTools	latest	Inspect the component tree and find what is re-rendering.	<code>npm i -g react-devtools</code>

iOS needs a Mac. There is no way around this — Apple requires macOS to build iOS apps. **Plan for Android first and say so openly in Review 1.** If nobody on the team has a Mac, iOS becomes a documented future-work item, not a surprise in Review 3.

Your daily workflow

Step	Command
1. Start the app	<code>pnpm start</code> (then scan the QR code with Expo Go)
2. Run on the emulator	<code>pnpm android</code>

Step	Command
3. Run the web portals	<code>pnpm dev:web</code>
4. Open the component library	<code>pnpm storybook</code>
5. Run end-to-end tests	<code>pnpm test:e2e</code>

The single most important setup step for you

Test on a real, cheap phone — not the emulator. The emulator runs on your laptop's processor and hides every performance problem you actually need to find.

What	How
Get a low-end test device	Borrow any ₹8,000-class Android phone. This is your reference device.
Simulate a slow network	Android Studio → Extended Controls → Cellular → set to 3G
Simulate no network	Airplane mode on the real phone — this is your Review 2 demo
Check performance	Chrome → <code>chrome://inspect</code> while the app runs on the phone

How to actually use these for your reviews

- **Review 1 (wireframes):** build the clickable prototype in **Figma**. Free, and you can demo it on a phone by sharing the prototype link.
- **Review 2 (offline demo):** the whole demo is *airplane mode on a real phone*. Rehearse it — turn off WiFi and mobile data, not just one.
- **Review 3 (accessibility):** enable **TalkBack** on Android (Settings → Accessibility) and navigate the app with the screen off. This is your strongest demo of the entire project.

4. P3 — AI & Data Lead

You build: the diagnosis models, crash detection, voice, matching, and the analytics pipeline.

Install

Tool	Version	Purpose	Install
Python	3.12 — not the newest	The AI ecosystem. See the warning below.	<code>winget install Python.Python.3.12</code>
uv	latest	Python package manager. Dramatically faster than pip.	<code>pip install uv</code>
PyTorch	2.x	Trains the neural network models.	<code>uv pip install torch torchvision</code>
scikit-learn + LightGBM	latest	Does most of the real work — matching, fraud, maintenance.	<code>uv pip install scikit-learn lightgbm pandas numpy</code>
JupyterLab	latest	Experiment notebooks. Where model work actually happens.	<code>uv pip install jupyterlab</code>
MLflow	latest	Tracks every experiment: parameters, metrics, and which model was best.	<code>uv pip install mlflow</code>
FastAPI	latest	Serves your models to P1's backend.	<code>uv pip install fastapi uvicorn</code>
ONNX Runtime	latest	Converts models to run on phones.	<code>uv pip install onnx onnxruntime</code>
Label Studio	latest	Label your training images and audio. Free and self-hosted.	<code>uv pip install label-studio</code>

⚠ Install Python 3.12, not 3.13 or 3.14

The newest Python release always arrives **months before** PyTorch and TensorFlow publish compatible packages. On the newest version, `pip install torch` simply fails with no useful error, and you will lose a day to it. NumPy and scikit-learn will install fine, which makes the problem look random. **Use 3.12.** If you already have a newer Python, install 3.12 alongside it and select it explicitly with `py -3.12 -m venv .venv`.

Your daily workflow

Step	Command
1. Activate the environment	<code>.venv\Scripts\activate</code>
2. Open notebooks	<code>jupyter lab</code>
3. Track experiments	<code>mlflow ui</code> → open <code>http://localhost:5000</code>
4. Serve models to the backend	<code>uvicorn app.main:app --reload --port 8000</code>
5. Label training data	<code>label-studio start</code>

Training models without a GPU

Your laptop almost certainly cannot train these models. **This is completely normal and costs you nothing.**

Platform	Free allowance	Best used for
Google Colab	~12 GB GPU, several hours per session	Image diagnosis, crash detection — your main training environment
Kaggle Notebooks	30 GPU hours per week	Longer training runs; more generous than Colab
Hugging Face	Free model + dataset hosting	Storing your trained models so the team can download them

Workflow: experiment locally on a small data sample in Jupyter → train the full model on Colab → download the trained file → serve it locally with FastAPI.

How to actually use these for your reviews

- **Review 1:** you deliver *analysis*, not models. Use Jupyter to profile the data and produce the honest "which of the 9 systems are real in v1" table.
- **Review 2 (proving a model works):** MLflow gives you the before/after comparison against your baseline automatically. **Screenshot the MLflow comparison view** — it is far more convincing to a panel than a number on a slide.
- **Review 3 (on-device):** export with ONNX, then report the model size and inference time measured **on the actual phone**, not on your laptop.

5. P4 — DevOps, QA & Security Lead

You build: the pipeline, the infrastructure, all testing, and security.

Install

Tool	Version	Purpose	Install
GitHub Actions	—	Automatic build and test on every pull request. Nothing to install — it is a file in the repository.	<code>.github/workflows/ci.yml</code>
kubect	latest	Controls Kubernetes.	<code>winget install Kubernetes.kubect</code>
k3d or minikube	latest	Runs Kubernetes on your laptop. k3d is much lighter.	<code>winget install k3d</code>
Terraform	1.9+	Defines cloud infrastructure as code.	<code>winget install Hashicorp.Terraform</code>
k6	latest	Load testing. Write the test in JavaScript.	<code>winget install k6</code>
OWASP ZAP	latest	Automated security scanning.	<code>winget install ZAP.ZAP</code>

Tool	Version	Purpose	Install
Grafana + Prometheus	—	Dashboards and metrics. Run in Docker, do not install.	via <code>docker compose</code>

Free cloud accounts to create in Week 1

Service	Free tier	Used for
GitHub	Unlimited private repositories, 2,000 CI minutes/month	Code, CI, project board, releases
Render or Railway	Free web service + free Postgres	Hosting the live demo for Review 3
Grafana Cloud	10,000 metrics, 50 GB logs	Monitoring without running a server
Sentry	5,000 errors/month	Catching crashes in the mobile app
Cloudflare	Free CDN, DNS and TLS	Domain and HTTPS for the demo

Start the SMS provider signup in Week 1. Indian SMS providers (Gupshup, Exotel, MSG91) require **TRAI DLT registration**, which takes weeks and cannot be rushed. Nothing else in this project has a lead time you cannot compress. Use their free sandbox for development while the registration is processing.

Your daily workflow

Step	Command
1. Start the local cluster	<code>k3d cluster create roadassist</code>
2. Deploy	<code>kubectl apply -f k8s/</code>
3. Watch the logs	<code>kubectl logs -f deploy/api</code>
4. Run a load test	<code>k6 run tests/load/booking.js</code>
5. Run a security scan	<code>zap-cli quick-scan http://localhost:3000</code>
6. Check the pipeline	GitHub → Actions tab

Your first real deliverable — the CI file

This is what blocks bad code from ever reaching the main branch. Create `.github/workflows/ci.yml`:

```
name: CI
on: [pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test
      - uses: gitleaks/gitleaks-action@v2 # blocks committed passwords
```

How to actually use these for your reviews

- Review 1:** your demo is this CI file *rejecting* a bad pull request. Deliberately commit a lint error, a failing test and a fake password, then show all three being blocked.
- Review 2:** show one request traced across services in Grafana. Visual, immediate, and it proves the system is genuinely integrated.
- Review 3:** run k6 live and show the graph. Then break something on purpose and show it recovering. **A demo where something fails and the system handles it is far stronger than one where nothing goes wrong.**

6. If Your Laptop Is Weak

Do not fight a 4 GB machine. Swap to the cloud version — all free.

Instead of running locally	Use this	Why it is better
PostgreSQL in Docker	Neon or Supabase (free Postgres)	No RAM cost, and the whole team shares one database
Android emulator	Expo Go on a real phone	Faster, and it is the honest test anyway
Training models locally	Google Colab / Kaggle	Free GPU, far faster than any student laptop
Local Kubernetes	Render / Railway free tier	Skip cluster management entirely for a project this size
Local Grafana	Grafana Cloud free tier	No container running in the background
VS Code + heavy extensions	GitHub Codespaces (60 free hours/month)	The full environment runs in the browser

Minimum realistic laptop spec per person: 8 GB RAM, 256 GB SSD. Below that, move to the cloud column above rather than struggling.

7. Working Together

Purpose	Tool	How you use it
Code	GitHub	One repository. Branch per feature. Pull request for every change.
Tasks	GitHub Projects	Free Kanban board built into the repository. Columns: Backlog → In Progress → Review → Done.
Daily communication	Discord or Slack free	Channels: <code>#general</code> , <code>#backend</code> , <code>#frontend</code> , <code>#ai</code> , <code>#alerts</code>
Design	Figma	P2 owns the file; everyone can view and comment.
Documents	Google Drive	Reports, review slides, meeting notes.
Diagrams	Mermaid (in Markdown) or draw.io	Mermaid lives in the repository and can be diffed — prefer it.
Screen recording	OBS Studio (free)	Record every review demo as a backup. Do this — live demos fail.

The Git workflow all four of you follow

```
git checkout main
git pull                                # always start from the latest code
git checkout -b feat/r2-p1-booking-api  # type / review / person / feature
# ... do your work ...
git add .
git commit -m "feat(booking): add booking creation endpoint"
git push -u origin feat/r2-p1-booking-api
# then open a Pull Request on GitHub and request a review
```

Four rules, no exceptions: 1. Never commit directly to `main` . Ever. 2. Every pull request needs one other person's approval. 3. Nobody merges their own code. 4. Pull the latest `main` before starting anything new.

8. Software for the Reviews Themselves

Deliverable	Tool	Notes
Presentation slides	Google Slides or Canva	Google Slides works simultaneously for all four people
Project report	Google Docs or Overleaf (LaTeX)	Overleaf if your institution requires IEEE format
Architecture diagrams	draw.io (free) or Mermaid	Export as PNG for slides

Deliverable	Tool	Notes
ER diagram	dbdiagram.io	Cleanest output for a database diagram
Demo recording	OBS Studio	Record every demo in advance as a backup
Screenshots	ShareX (free)	Better than the built-in snipping tool for annotation
Poster	Canva	Free templates

The single most valuable habit: record every demo the day *before* the review. Live demos fail — the WiFi drops, the phone does not pair, an API times out. A recorded backup turns a disaster into a five-second recovery.

9. Day-1 Setup Checklist

Work top to bottom. Do not skip ahead — later steps depend on earlier ones.

#	Who	Task	Time
1	All	Install the shared toolchain (Section 1)	60 min
2	All	Create GitHub accounts; P4 creates the repository and adds everyone	15 min
3	All	<code>git clone</code> the repository; run <code>pnpm install</code>	10 min
4	All	Install VS Code extensions; commit the shared settings file	15 min
5	P4	Create the CI file; verify it runs on a test pull request	45 min
6	P4	Start the SMS provider / DLT registration — longest lead time in the project	30 min
7	P1	Create <code>docker-compose.yml</code> ; confirm Postgres, Redis and Kafka all start	45 min
8	P2	Install Android Studio and Expo Go; run a blank app on a real phone	90 min
9	P3	Install Python 3.12 in a virtual environment; verify <code>import torch</code> works	45 min
10	All	Everyone pushes one trivial pull request and reviews someone else's	30 min

Step 10 matters more than it looks. If all four of you have successfully pushed and reviewed a pull request on Day 1, the workflow is proven before there is any pressure on it.

10. Windows Problems You Will Hit

Problem	Cause	Fix
Docker Desktop will not start	WSL 2 not enabled	Run <code>wsl --install</code> in an Administrator PowerShell, then restart
<code>pnpm</code> not recognised after install	PATH not refreshed	Close and reopen the terminal. Always.
A newly installed tool is blocked on first run	Windows Smart App Control	Right-click the file → Properties → tick Unblock
<code>pip install torch</code> fails with no clear error	Python version too new	Install Python 3.12 and use <code>py -3.12 -m venv .venv</code>
Every file shows as changed in a pull request	Line-ending mismatch	<code>git config --global core.autocrlf false</code> and set <code>"files.eol": "\n"</code> in VS Code
Editing files with PowerShell corrupts accented characters	<code>Set-Content</code> defaults to the system codepage, not UTF-8	Edit in VS Code, or pass <code>-Encoding utf8</code> explicitly. Never bulk-edit source files with <code>Get-Content -Raw \</code> <code>Set-Content</code> .
Android emulator extremely slow	Hardware acceleration off	Enable virtualisation in the BIOS — or just use a real phone with Expo Go
Port 5432 already in use	A native PostgreSQL is already running	Stop the Windows service, or change the Docker port to <code>5433:5432</code>

11. Total Cost

Category	Cost
All development software	₹0
GitHub (private repositories + CI)	₹0
Cloud hosting for the demo	₹0 (free tiers)
GPU for model training	₹0 (Colab / Kaggle)
Monitoring and error tracking	₹0 (free tiers)
SMS testing	₹0 (provider sandbox)
Total	₹0

The only genuine costs are optional: a domain name (about ₹800/year) and a Google Play developer account (a one-time \$25) if you want to publish the app publicly. **Neither is needed to pass any of the three reviews.**